

**VISVESVARAYA TECHNOLOGICAL UNIVERSITY,
BELAGAVI**

(Established by the Government of Karnataka)

A PROJECT REPORT

ON

**DESIGN AND SIMULATION OF A
32-BIT SINGLE-CYCLE RISC-V PROCESSOR
USING VERILOG HDL**

Submitted in partial fulfillment of the requirements for the award of the Degree of

BACHELOR OF ENGINEERING

IN

ELECTRONICS AND COMMUNICATION ENGINEERING

Submitted by

RAJENDRA NAIK

USN:4UB23EC042

**DEPARTMENT OF ELECTRONICS AND
COMMUNICATION ENGINEERING**

UNIVERSITY B.D.T. COLLEGE OF ENGINEERING

DAVANAGERE – 577004, KARNATAKA

(A Constituent College of Visvesvaraya Technological University, Belagavi)

ACADEMIC YEAR 2026–2027

ABSTRACT

The increasing demand for efficient, flexible, and customizable processors has accelerated the adoption of open-source Instruction Set Architectures (ISAs) in modern computing systems. Among the available open-source architectures, **RISC-V** has emerged as a promising solution due to its modular design, scalability, and freedom from licensing restrictions. Its simplicity and flexibility make it suitable for academic research, embedded systems, Internet of Things (IoT) applications, and next-generation computing platforms.

This project presents the **design and simulation of a 32-bit Single-Cycle RISC-V Processor implementing the RV32I instruction set using Verilog Hardware Description Language (HDL)**. The processor is developed using a modular approach, where each functional unit is designed, simulated, and verified independently before integration into a complete processor architecture. The major modules include the **Program Counter (PC), Instruction Memory, Register File, Immediate Generator, Control Unit, ALU Control Unit, Arithmetic Logic Unit (ALU), Data Memory, and Top Module**. These modules collectively perform instruction fetch, decode, execution, memory access, and register write-back operations within a single clock cycle.

The processor supports the execution of essential **RV32I instructions**, including arithmetic, logical, load/store, and branch operations. The design is implemented in **Verilog HDL** and functionally verified using **AMD Xilinx Vivado 2025.2**. Behavioral simulation is carried out to validate the correct operation of each module and the integrated processor. The generated **simulation waveforms** confirm the proper execution of instructions, while the **Register Transfer Level (RTL) schematic** verifies the structural correctness of the processor architecture and interconnection of modules.

The proposed processor provides a simple yet effective platform for understanding processor organization, digital hardware design, and computer architecture. Its modular structure improves readability, simplifies debugging, and allows future enhancements such as pipelining, cache memory, interrupt handling, additional RISC-V instruction extensions, and FPGA implementation. The successful simulation results demonstrate that the designed processor correctly executes the supported RV32I instructions and serves as a strong educational and research platform for learning modern processor design using open-source technologies.

CHAPTER 1

INTRODUCTION

1.1 Introduction

Modern computing systems rely on processors to perform arithmetic, logical, control, and data processing operations. A processor, also known as the Central Processing Unit (CPU), is considered the core component of any digital system because it executes program instructions and coordinates communication between hardware components such as memory and input/output devices. Processors are widely used in personal computers, smartphones, embedded systems, industrial automation, automotive electronics, medical devices, robotics, and Internet of Things (IoT) applications.

Over the past few decades, processor architectures have evolved significantly to meet the growing demand for higher performance, lower power consumption, and greater flexibility. Most commercial processors are based on proprietary Instruction Set Architectures (ISAs), such as x86 and ARM. Although these architectures provide excellent performance, they require licensing agreements for commercial implementation and limit customization. These restrictions create challenges for educational institutions, researchers, and small-scale developers who wish to study or design processor architectures.

To overcome these limitations, the **RISC-V Instruction Set Architecture (ISA)** was introduced as an open-standard, royalty-free processor architecture developed at the University of California, Berkeley. RISC-V follows the principles of **Reduced Instruction Set Computing (RISC)**, which emphasizes a simplified instruction set, fixed instruction formats, and efficient execution. Its modular architecture allows designers to implement only the required instruction extensions, making it suitable for applications ranging from low-power embedded systems to high-performance computing platforms.

The RISC-V architecture consists of a base integer instruction set and optional extensions for multiplication, floating-point operations, atomic instructions, compressed instructions, and vector processing. Because of its openness and scalability, RISC-V has gained widespread acceptance in academia, industry, and research communities. It provides an ideal platform for learning processor architecture and digital hardware design while encouraging innovation in processor development.

A **Single-Cycle Processor** is one of the simplest processor architectures in which every instruction is completed during a single clock cycle. All stages of instruction execution—including instruction fetch, instruction decode, execution, memory access, and write-back—are performed within one clock period. Although the clock period is relatively long compared to pipelined processors, the architecture is simple to design, easy to understand, and suitable for educational purposes.

In this project, a **32-bit Single-Cycle RISC-V Processor** implementing the **RV32I base instruction set** is designed and simulated using **Verilog Hardware Description Language (HDL)**. The processor is developed using a modular methodology consisting of the **Program Counter (PC)**, **Instruction Memory**, **Register File**, **Immediate Generator**, **Control Unit**, **ALU Control Unit**, **Arithmetic Logic Unit (ALU)**, **Data**

Memory, and Top Module. Each module is individually designed and verified before being integrated into the complete processor architecture.

The processor supports arithmetic, logical, memory access, and branch instructions of the RV32I instruction set. Functional verification is performed through behavioral simulation using **AMD Xilinx Vivado 2025.2**, and the generated Register Transfer Level (RTL) schematic confirms the structural correctness of the design. The modular architecture adopted in this project enables easy debugging, future enhancements, and possible FPGA implementation.

The successful implementation of this project demonstrates the practicality of open-source processor development and provides a strong foundation for understanding computer architecture, digital logic design, and hardware description languages. Furthermore, the proposed processor can be extended with advanced features such as pipelining, cache memory, interrupt handling, and additional RISC-V instruction extensions, making it suitable for future research and development in embedded systems and VLSI design.

1.2 Problem Statement

The rapid advancement of embedded systems, Internet of Things (IoT), artificial intelligence, and edge computing has created a growing demand for processors that are efficient, flexible, and cost-effective. Most commercially available processors are based on proprietary Instruction Set Architectures (ISAs), such as x86 and ARM, which require licensing agreements and restrict customization. These limitations make them less suitable for academic learning, research, and the development of application-specific processors.

Although open-source processor architectures have gained popularity, many existing processor implementations are complex due to features such as pipelining, cache memory, branch prediction, and superscalar execution. Such complexity increases hardware resource utilization and makes it difficult for students and beginners to understand the internal working of a processor. Therefore, there is a need for a simple processor architecture that clearly demonstrates instruction execution, datapath organization, and control signal generation.

The RISC-V Instruction Set Architecture (ISA) provides an open-standard and royalty-free platform for processor development. However, designing a processor based on the RV32I instruction set requires careful implementation of various functional modules, including the Program Counter, Instruction Memory, Register File, Immediate Generator, Control Unit, Arithmetic Logic Unit (ALU), and Data Memory. Proper integration and verification of these modules are essential to ensure correct execution of arithmetic, logical, memory, and branch instructions.

Hence, this project aims to design and simulate a **32-bit Single-Cycle RISC-V Processor using Verilog HDL**. The processor is developed using a modular architecture and functionally verified through behavioral simulation in **AMD Xilinx Vivado 2025.2**. The

proposed design provides a simple, reliable, and educational platform for understanding processor architecture while serving as a foundation for future enhancements such as pipelining, cache memory, interrupt handling, and FPGA implementation.

1.3 Objectives

The main objectives of the proposed project are:

- To study the architecture and working principles of the RISC-V Instruction Set Architecture (ISA).
- To design a **32-bit Single-Cycle RISC-V Processor** implementing the RV32I instruction set.
- To develop the processor using **Verilog Hardware Description Language (HDL)** with a modular design approach.
- To implement key processor modules, including the Program Counter, Instruction Memory, Register File, Immediate Generator, Control Unit, ALU Control Unit, Arithmetic Logic Unit (ALU), and Data Memory.
- To integrate all processor modules into a complete processor architecture.
- To perform behavioral simulation and functional verification using **AMD Xilinx Vivado 2025.2**.
- To generate and analyze the Register Transfer Level (RTL) schematic of the processor.
- To validate the correct execution of arithmetic, logical, load/store, and branch instructions of the RV32I instruction set.
- To provide a scalable processor design that can be extended for future FPGA implementation and advanced processor architectures.

1.4 Scope of the Study

The scope of this project is limited to the design, implementation, and simulation of a **32-bit Single-Cycle RISC-V Processor** based on the **RV32I base instruction set** using Verilog HDL. The processor is developed using a modular architecture in which each functional block is designed and verified independently before integration into the complete processor.

The processor supports fundamental instruction categories, including arithmetic operations, logical operations, load/store instructions, and branch instructions. Functional verification is carried out using behavioral simulation in **AMD Xilinx Vivado 2025.2**, and the generated RTL schematic is used to verify the structural correctness of the design.

This project focuses only on simulation and functional verification. Advanced features such as pipelining, cache memory, branch prediction, interrupt handling, floating-point

operations, multicore architecture, and FPGA hardware implementation are outside the scope of the present work. However, the modular design methodology adopted in this project allows these features to be incorporated in future enhancements.

The proposed processor serves as an educational platform for understanding processor organization, digital system design, and hardware description languages. It also provides a strong foundation for future research and development in embedded systems, VLSI design, and computer architecture.

1.5 Organization of the Report

This report is organized into five chapters to provide a systematic presentation of the proposed **32-bit Single-Cycle RISC-V Processor**.

Chapter 1 – Introduction presents the background of processor architecture, the RISC-V Instruction Set Architecture, the problem statement, objectives, scope of the study, and the overall organization of the report.

Chapter 2 – Literature Survey reviews previous research related to RISC-V processors, processor architectures, and hardware implementation techniques. It summarizes the contributions of existing research and identifies the motivation for the proposed work.

Chapter 3 – System Design and Methodology describes the proposed processor architecture, block diagram, working principle, processor modules, RV32I instruction formats, processor datapath, algorithm, and software tools used for implementation and simulation.

Chapter 4 – Implementation and Results explains the implementation of each processor module using Verilog HDL, presents the behavioral simulation results, RTL schematic, waveform analysis, functional verification, and discusses the performance of the proposed processor.

Chapter 5 – Conclusion and Future Scope summarizes the achievements of the project, highlights the successful implementation and verification of the processor, and discusses possible future enhancements such as pipelining, cache memory, interrupt handling, additional RISC-V instruction extensions, and FPGA implementation.

CHAPTER 02

LITERATURE SURVEY

1. K. Asanović and A. Waterman, *"The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA"*

Asanović and Waterman introduced the **RISC-V Instruction Set Architecture (ISA)** as an open-source and royalty-free processor architecture. The manual defines the **RV32I base instruction set**, instruction formats, register organization, memory operations, and execution model. The architecture is designed to be simple, modular, and scalable, making it suitable for embedded systems, academic research, and industrial applications. The RV32I instruction set described in this manual forms the basis of the proposed processor.

2. A. Waterman, Y. Lee, D. A. Patterson, and K. Asanović, *"The Rocket Chip Generator"*

Waterman et al. developed the **Rocket Chip Generator**, a configurable RISC-V processor generator that enables designers to create customized processor cores for different applications. The processor supports modular hardware design, configurable cache memory, and pipelined execution. Their work demonstrated that RISC-V processors can be efficiently implemented on FPGA and ASIC platforms while providing flexibility and scalability for future enhancements.

3. C. Wolf, *"PicoRV32: A Size-Optimized RISC-V CPU"*

Clifford Wolf proposed **PicoRV32**, a compact and resource-efficient RISC-V processor implementing the RV32I instruction set. The processor is optimized for FPGA and ASIC applications, requiring very low hardware resources while maintaining good performance. Its simple architecture makes it suitable for embedded systems and educational applications where hardware efficiency and ease of implementation are important.

4. D. A. Patterson and A. Waterman, *"The RISC-V Reader: An Open Architecture Atlas"*

Patterson and Waterman explained the concepts of the **RISC-V architecture** in a simple and practical manner. The book discusses processor organization, instruction execution, memory operations, and register usage. It highlights the advantages of an open-source ISA over proprietary architectures and provides a strong foundation for learning processor design and computer architecture.

5. C. Celio, D. A. Patterson, and K. Asanović, "*The Berkeley Out-of-Order Machine (BOOM): An Open-Source High-Performance RISC-V Processor*"

Celio et al. developed the **Berkeley Out-of-Order Machine (BOOM)**, a high-performance RISC-V processor that uses advanced techniques such as pipelining, branch prediction, and out-of-order execution. The processor improves execution speed and overall performance while maintaining compatibility with the RISC-V ISA. The work demonstrates the scalability of RISC-V for advanced computing applications.

6. D. A. Patterson and K. Asanović, "*Instruction Sets Should Be Free: The Case for RISC-V*"

Patterson and Asanović discussed the importance of open-source instruction set architectures and introduced RISC-V as a flexible and extensible processor architecture. They explained how an open ISA reduces development costs, encourages innovation, and supports research and education. Their work established RISC-V as a strong alternative to proprietary processor architectures and motivated the development of open-source processor designs.

CHAPTER 03

SYSTEM DESIGN AND METHODOLOGY

3.1 Introduction

This chapter describes the design and implementation methodology of the proposed **32-bit Single-Cycle RISC-V Processor**. The processor is designed using **Verilog Hardware Description Language (HDL)** and functionally verified using **AMD Xilinx Vivado 2025.2**. A modular design approach is adopted, where each processor component is developed independently and later integrated into a complete processor architecture.

The proposed processor implements the **RV32I base instruction set** and executes instructions in a single clock cycle. The architecture consists of the **Program Counter (PC), Instruction Memory, Register File, Immediate Generator, Control Unit, ALU Control Unit, Arithmetic Logic Unit (ALU), Data Memory, and Top Module**. These modules work together to perform instruction fetch, decode, execution, memory access, and write-back operations.

The processor supports arithmetic, logical, load/store, and branch instructions. Behavioral simulation and RTL schematic generation are performed to verify the functionality and correctness of the design. The modular architecture also allows future enhancements such as pipelining, cache memory, and FPGA implementation.

3.2 Proposed System

The proposed system is a **32-bit Single-Cycle RISC-V Processor** based on the **RV32I Instruction Set Architecture (ISA)**. It is designed to execute instructions within a single clock cycle, making the processor simple, efficient, and suitable for educational and research purposes.

The processor fetches instructions from the instruction memory, decodes them using the control unit, reads operands from the register file, performs arithmetic or logical operations using the ALU, accesses data memory for load/store instructions, and writes the result back to the destination register. Each module performs a specific function, ensuring a modular and organized processor design.

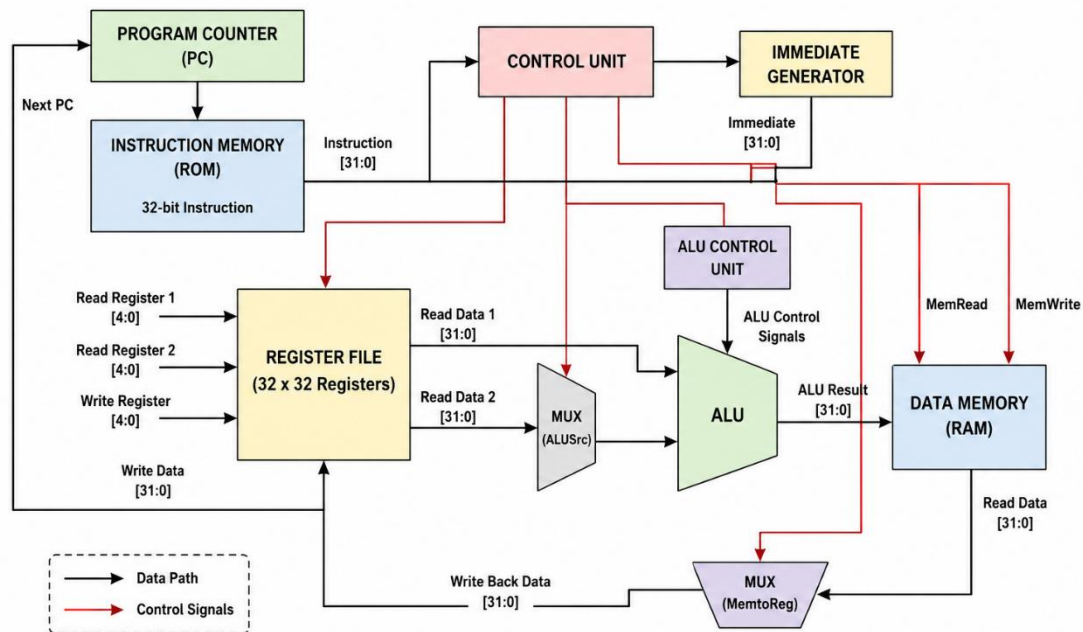
The entire processor is implemented using **Verilog HDL**, and its functionality is verified through behavioral simulation in **AMD Xilinx Vivado 2025.2**. The modular design improves readability, simplifies debugging, and provides flexibility for future enhancements.

3.3 Block Diagram

The architecture of the proposed **32-bit Single-Cycle RISC-V Processor** consists of several functional modules that work together to execute instructions. The **Program Counter (PC)** stores the address of the current instruction and updates it every clock

cycle. The **Instruction Memory** stores the program instructions and provides the required instruction to the processor. The **Control Unit** decodes the instruction and generates control signals for different modules. The **Register File** stores 32 general-purpose registers and supplies operands to the ALU. The **Immediate Generator** extracts and extends immediate values from the instruction. The **ALU Control Unit** determines the operation to be performed by the ALU based on the instruction type. The **Arithmetic Logic Unit (ALU)** performs arithmetic and logical operations such as addition, subtraction, AND, OR, and comparison. The **Data Memory** is used for load and store instructions. Finally, the **Top Module** integrates all individual modules into a complete processor for simulation and verification.

BLOCK DIAGRAM OF 32-bit SINGLE-CYCLE RISC-V PROCESSOR



3.4 Methodology

The proposed processor is developed using a modular design methodology. Each processor module is designed individually using **Verilog HDL** and tested through simulation before integrating it into the complete processor. Initially, the Program Counter, Instruction Memory, Register File, Immediate Generator, Control Unit, ALU Control Unit, ALU, and Data Memory are implemented separately. After verifying the functionality of each module, they are connected through the Top Module to form the complete 32-bit Single-Cycle RISC-V Processor.

The processor executes instructions according to the **RV32I Instruction Set Architecture**. During execution, the Program Counter fetches an instruction from Instruction Memory. The Control Unit decodes the instruction and generates the required control signals. The Register File provides the source operands to the ALU, while the Immediate Generator supplies immediate values whenever required. The ALU performs arithmetic or logical operations, and Data Memory is accessed for load and store instructions. Finally, the result is written back to the destination register. The complete processor is verified using behavioral simulation and RTL schematic generation in **AMD Xilinx Vivado 2025.2**.

3.5 Working Principle

The working of the proposed processor begins with the **Program Counter (PC)**, which points to the address of the current instruction. The Instruction Memory fetches the instruction, and the Control Unit decodes it to identify the required operation. Based on the instruction type, the Register File provides the necessary operands, while the Immediate Generator produces immediate data if required.

The ALU performs arithmetic or logical operations according to the control signals generated by the ALU Control Unit. For memory-related instructions, the processor accesses the Data Memory to read or write data. The computed result is then written back into the destination register of the Register File. After completing the instruction, the Program Counter is updated to fetch the next instruction. This sequence continues until all instructions in the program are executed. The entire operation is completed within a single clock cycle for each instruction.

3.6 Design of Individual Modules

3.6.1 Program Counter (PC)

The **Program Counter (PC)** is responsible for storing the address of the current instruction. During every clock cycle, it supplies the instruction address to the Instruction Memory. After an instruction is executed, the PC is updated to point to the next instruction, normally by adding 4 to the current address. For branch and jump instructions, the PC is updated with the target address. The Program Counter ensures that instructions are executed in the correct sequence.

3.6.2 Instruction Memory

The **Instruction Memory** stores the program instructions that are executed by the processor. It receives the instruction address from the Program Counter and returns the corresponding 32-bit instruction. These instructions are then sent to the Control Unit and other modules for decoding and execution. During simulation, the instruction memory is initialized with the test program.

3.6.3 Register File

The **Register File** contains **32 general-purpose registers**, each having a width of **32 bits**. It provides two read ports and one write port. The source operands are read from the registers specified by the instruction, while the execution result is written back to the destination register. Register **x0** always contains zero as defined by the RISC-V specification.

3.6.4 Immediate Generator

The **Immediate Generator** extracts the immediate value from different instruction formats such as **I-type, S-type, B-type, U-type, and J-type**. It performs sign extension whenever required and generates a 32-bit immediate value. This immediate value is used by the ALU during arithmetic operations, memory access, and branch instructions.

3.6.5 Control Unit

The **Control Unit** decodes the opcode of the fetched instruction and generates the required control signals for processor operation. These control signals determine ALU operations, register write enable, memory read/write operations, operand selection, and write-back control. The Control Unit acts as the central controller of the processor.

3.6.6 ALU Control Unit

The **ALU Control Unit** receives control information from the Control Unit along with the instruction function fields (funct3 and funct7). Based on these inputs, it generates the control signals required by the ALU to perform arithmetic and logical operations such as addition, subtraction, AND, OR, XOR, and comparison.

3.6.7 Arithmetic Logic Unit (ALU)

The **Arithmetic Logic Unit (ALU)** is the main computational unit of the processor. It performs arithmetic operations such as addition and subtraction, as well as logical operations including AND, OR, XOR, and comparison. The ALU executes the operation selected by the ALU Control Unit and sends the result either to the Register File or the Data Memory.

3.6.8 Data Memory

The **Data Memory** is used for load and store instructions. During a store operation, data from the Register File is written into memory. During a load operation, data is read from memory and written back to the destination register. Data Memory enables the processor to access external data required by the executing program.

3.6.9 Top Module

The **Top Module** integrates all processor modules, including the Program Counter, Instruction Memory, Register File, Immediate Generator, Control Unit, ALU Control Unit, ALU, and Data Memory. It establishes the connections between these modules and forms the complete **32-bit Single-Cycle RISC-V Processor**. The Top Module is used for simulation, functional verification, and RTL generation in **AMD Xilinx Vivado 2025.2**.

3.7 Design Flow

The design of the proposed **32-bit Single-Cycle RISC-V Processor** follows a systematic approach to ensure correct functionality and easy verification. Initially, the **RV32I Instruction Set Architecture (ISA)** is studied to understand the instruction formats and processor operations. Based on these specifications, the processor architecture is divided into individual modules such as the Program Counter, Instruction Memory, Register File, Immediate Generator, Control Unit, ALU Control Unit, ALU, and Data Memory.

Each module is implemented separately using **Verilog HDL** and verified through behavioral simulation in **AMD Xilinx Vivado 2025.2**. After confirming the functionality of every module, they are integrated into a **Top Module** to form the complete processor. A testbench is then created to provide clock, reset, and test instructions for simulation. Finally, behavioral simulation, waveform analysis, and RTL schematic generation are performed to verify the correct operation of the processor.

Design Flow Steps:

1. Design the processor architecture.
2. Develop individual modules using Verilog HDL.
3. Simulate and verify each module.
4. Integrate all modules into the Top Module.
5. Create the testbench.
6. Perform behavioral simulation.
7. Analyze simulation waveforms.
8. Generate the RTL schematic.
9. Verify the complete processor functionality.

3.9 Software Requirements

The processor is designed and verified using industry-standard software tools.

Software Requirements:

- AMD Xilinx Vivado Design Suite 2025.2
-
-

-
-
- Verilog Hardware Description Language (HDL)
 - Vivado Simulator (XSim)
-

3.10 Advantages of the Proposed System

The proposed **32-bit Single-Cycle RISC-V Processor** offers several advantages for learning and processor development:

- Open-source RISC-V architecture.
 - Simple and modular processor design.
 - Easy to understand and implement.
 - Supports the RV32I base instruction set.
 - Faster instruction execution in a single clock cycle.
 - Easy verification using behavioral simulation.
 - Suitable for educational and research applications.
 - Can be extended with pipelining, cache memory, and FPGA implementation in the future.
-
-

CHAPTER 04

IMPLEMENTATION AND RESULTS

4.1 Introduction

This chapter presents the implementation and verification of the proposed **32-bit Single-Cycle RISC-V Processor**. The processor is designed using **Verilog Hardware Description Language (HDL)** and simulated using **AMD Xilinx Vivado 2025.2**. Each module is implemented separately and tested to ensure correct functionality before integrating it into the complete processor. After integration, the processor is verified through behavioral simulation, waveform analysis, and RTL schematic generation. The simulation results confirm that the processor correctly executes the supported **RV32I instructions**.

4.2 Implementation of Individual Modules

4.2.1 Program Counter (PC)

The Program Counter (PC) module is implemented to store the address of the current instruction. It updates the instruction address on every clock cycle and resets to zero when the reset signal is activated. During simulation, the PC correctly increments the address by 4 after each instruction, ensuring sequential execution.

4.2.2 Instruction Memory

The Instruction Memory module stores the program instructions. It receives the instruction address from the Program Counter and outputs the corresponding 32-bit instruction. The instruction memory is initialized with test instructions for simulation, allowing the processor to fetch instructions correctly.

4.2.3 Register File

The Register File consists of **32 general-purpose registers**, each 32 bits wide. It supports two read ports and one write port, allowing simultaneous reading of source operands and writing of the execution result. The register **x0** is permanently fixed to zero according to the RISC-V specification.

4.2.4 Immediate Generator

The Immediate Generator extracts immediate values from different instruction formats and performs sign extension whenever required. The generated immediate values are used by arithmetic, memory, and branch instructions during execution.

4.2.5 Control Unit

The Control Unit decodes the instruction opcode and generates the required control signals for all processor modules. These control signals determine ALU operations, memory access, register write operations, and operand selection.

4.2.6 ALU Control Unit

The ALU Control Unit generates the appropriate control signals for the ALU based on the opcode, **funct3**, and **funct7** fields of the instruction. It enables the ALU to perform the correct arithmetic or logical operation.

4.2.7 Arithmetic Logic Unit (ALU)

The ALU performs arithmetic and logical operations such as addition, subtraction, AND, OR, XOR, and comparison. It processes the input operands according to the control signals received from the ALU Control Unit and generates the output result.

4.2.8 Data Memory

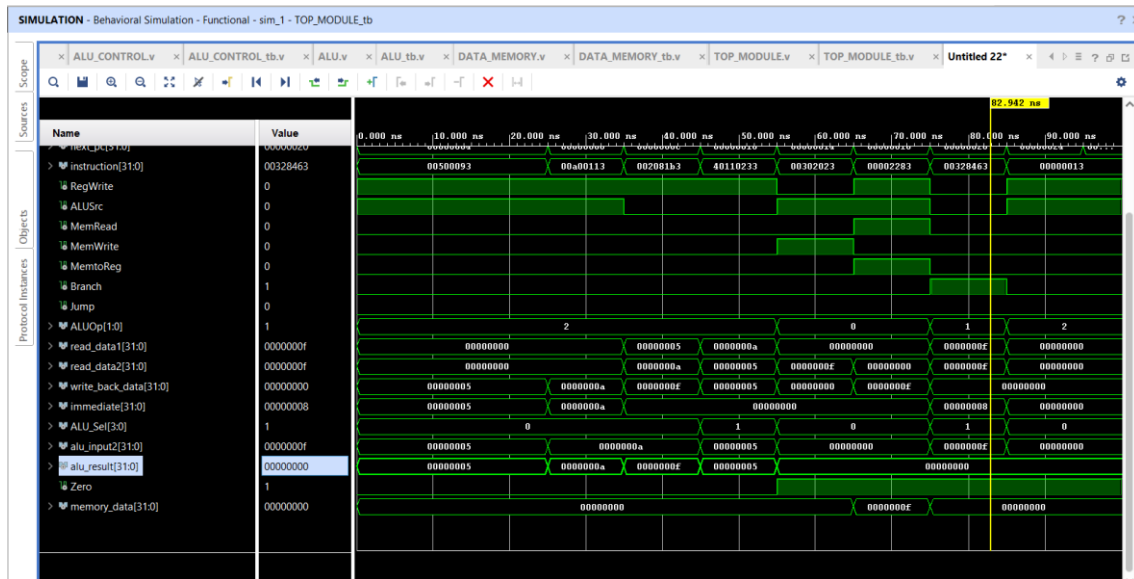
The Data Memory module stores data during store instructions and retrieves data during load instructions. It communicates with the Register File to complete memory read and write operations.

4.2.9 Top Module

The Top Module integrates all processor modules into a complete **32-bit Single-Cycle RISC-V Processor**. It establishes the data flow and control signals between modules, enabling the processor to execute instructions correctly during simulation.

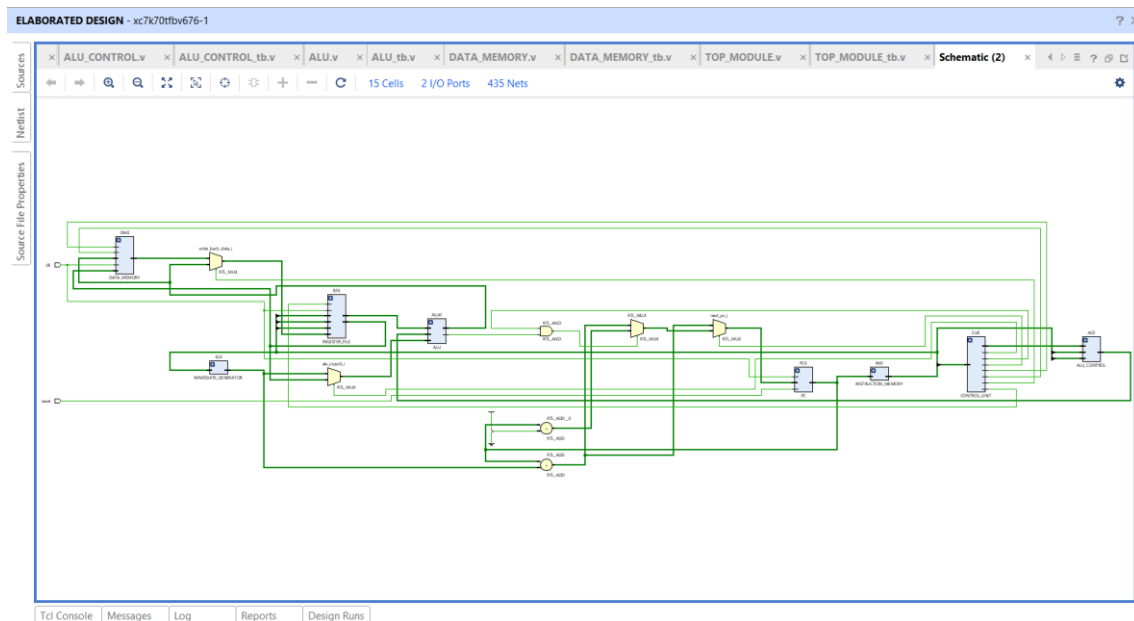
4.3 Behavioral Simulation Results

Behavioral simulation is performed using the **Vivado Simulator (XSim)** to verify the functionality of each processor module and the complete processor. The simulation confirms that the Program Counter updates correctly, instructions are fetched and decoded properly, ALU operations produce accurate results, and register values are updated as expected. Load and store instructions are also verified through Data Memory operations. The obtained waveforms confirm the correct execution of the supported RV32I instructions.



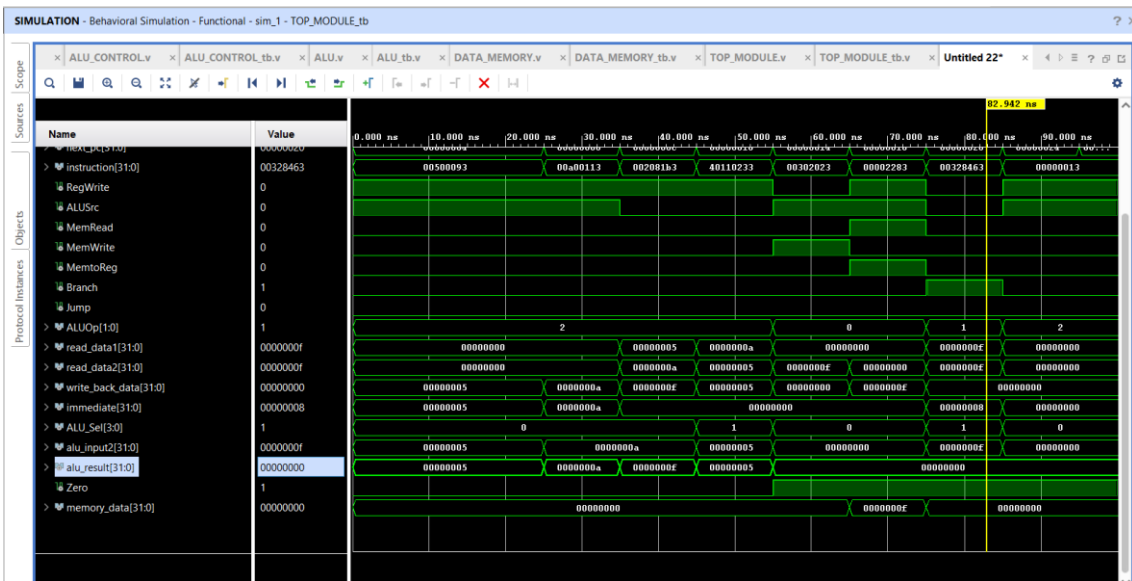
4.4 RTL Schematic

The RTL (Register Transfer Level) schematic is generated automatically by **AMD Xilinx Vivado 2025.2** after successful synthesis of the Verilog design. The schematic illustrates the interconnection of all processor modules, including the Program Counter, Instruction Memory, Register File, Control Unit, ALU, Data Memory, and other supporting blocks. The generated RTL confirms that the processor architecture has been implemented correctly.



4.5 Waveform Analysis

The simulation waveform verifies the timing and functionality of the processor. The clock signal controls the execution of instructions, while the reset signal initializes the processor. The waveform shows the Program Counter updating sequentially, instructions being fetched from memory, operands being read from the Register File, ALU operations producing correct outputs, and results being written back to the destination registers. These waveforms confirm the successful operation of the processor.



4.6 Results

The simulation results demonstrate that the proposed **32-bit Single-Cycle RISC-V Processor** operates correctly according to the **RV32I Instruction Set Architecture**. All processor modules function as expected, and the integrated processor successfully executes arithmetic, logical, load, store, and branch instructions. The modular implementation simplifies testing, debugging, and future enhancements. The project successfully meets its design objectives and provides a strong foundation for extending the processor with advanced features such as pipelining, cache memory, and FPGA implementation.

CHAPTER 05

CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

The proposed **32-bit Single-Cycle RISC-V Processor** was successfully designed and simulated using **Verilog HDL** in **AMD Xilinx Vivado 2025.2**. The processor was developed based on the **RV32I Instruction Set Architecture (ISA)** using a modular design approach. Individual modules, including the Program Counter, Instruction Memory, Register File, Immediate Generator, Control Unit, ALU Control Unit, Arithmetic Logic Unit (ALU), and Data Memory, were implemented and integrated into a complete processor.

Behavioral simulation and RTL verification confirmed that the processor correctly executes arithmetic, logical, load, store, and branch instructions. The simulation results demonstrated the correct functionality of each module and the overall processor. The modular architecture simplified implementation, testing, and debugging, making the processor suitable for educational and research purposes. Thus, the objectives of designing and verifying a 32-bit Single-Cycle RISC-V Processor were successfully achieved.

5.2 Future Scope

The proposed processor can be further enhanced by incorporating advanced features to improve its performance and functionality. Some possible future improvements include:

- Implementation of a **pipelined RISC-V processor** to increase execution speed.
- Integration of **instruction and data cache memory** to reduce memory access time.
- Support for additional **RISC-V instruction set extensions** such as multiplication, floating-point, and compressed instructions.
- FPGA implementation and hardware validation using AMD Xilinx FPGA development boards.
- Addition of interrupt handling and exception management.
- Development of a multi-core RISC-V processor for parallel processing applications.
- Integration with AI and embedded systems for real-time applications.
- Performance optimization through advanced processor design techniques.

These enhancements will improve the processor's efficiency, scalability, and suitability for modern embedded and high-performance computing applications.

REFERENCES

- [1] A. Waterman, Y. Lee, D. A. Patterson, and K. Asanović, *The RISC-V Instruction Set Manual, Volume I: User-Level ISA*, Version 2.1, EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2016-118, May 2016.
 - [2] K. Asanović, Y. Lee, A. Waterman, D. A. Patterson, and C. Celio, *The Rocket Chip Generator*, EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2016-17, Apr. 2016.
 - [3] C. Wolf, "PicoRV32: A Size-Optimized RISC-V CPU," Open-Source Processor Project, 2016.
 - [4] D. A. Patterson and A. Waterman, *The RISC-V Reader: An Open Architecture Atlas*, 1st ed., Strawberry Canyon LLC, Berkeley, CA, USA, 2017.
 - [5] C. Celio, P. Dabbelt, Y. Lee, D. A. Patterson, and K. Asanović, "The Berkeley Out-of-Order Machine (BOOM): An Open-Source High-Performance RISC-V Processor," EECS Department, University of California, Berkeley, 2015.
 - [6] D. A. Patterson and K. Asanović, *Instruction Sets Should Be Free: The Case for RISC-V*, EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2014-146, Aug. 2014.
-
-

APPENDIX

The appendix contains the supporting materials related to the implementation and verification of the proposed **32-bit Single-Cycle RISC-V Processor**. It includes the Verilog HDL source code, testbench code, behavioral simulation waveforms, and RTL schematic for each module. These materials provide additional technical details and validate the functionality of the proposed processor.

1 Program Counter (PC)

Verilog Code:

```
module PC

input      clk,

input      reset,

input  [31:0] next_pc,

output reg [31:0] pc

);

always @(posedge clk or posedge reset)

begin

    if (reset)

        pc <= 32'h00000000;

    else

        pc <= next_pc;

end

endmodule
```

Verilog Test_bench:

```
module PC_tb;

reg clk;

reg reset;

reg [31:0] next_pc;

wire [31:0] pc;

PC uut (

    .clk(clk),
```

```

.reset(reset),

.next_pc(next_pc),

.pc(pc)
);

always #5 clk = ~clk;

initial

begin

    clk = 1'b0;

    reset = 1'b1;

    next_pc = 32'h00000000;

    #20;

    reset = 1'b0;

    next_pc = 32'h00000004;

    #10;

    next_pc = 32'h00000008;

    #10;

    next_pc = 32'h0000000C;

    #10;

    next_pc = 32'h00000010;

    #10;

    next_pc = 32'h00000014;

    #10;

    next_pc = 32'h00000018;

    #10;

    $finish;

End;

initial

begin

    $display("-----");

    $display("Time\tReset\tNext_PC\t\tPC");

    $display("-----");

```

```
$monitor("%0t\t%b\t%h\t%h",
```

```
$time,
```

```
reset,
```

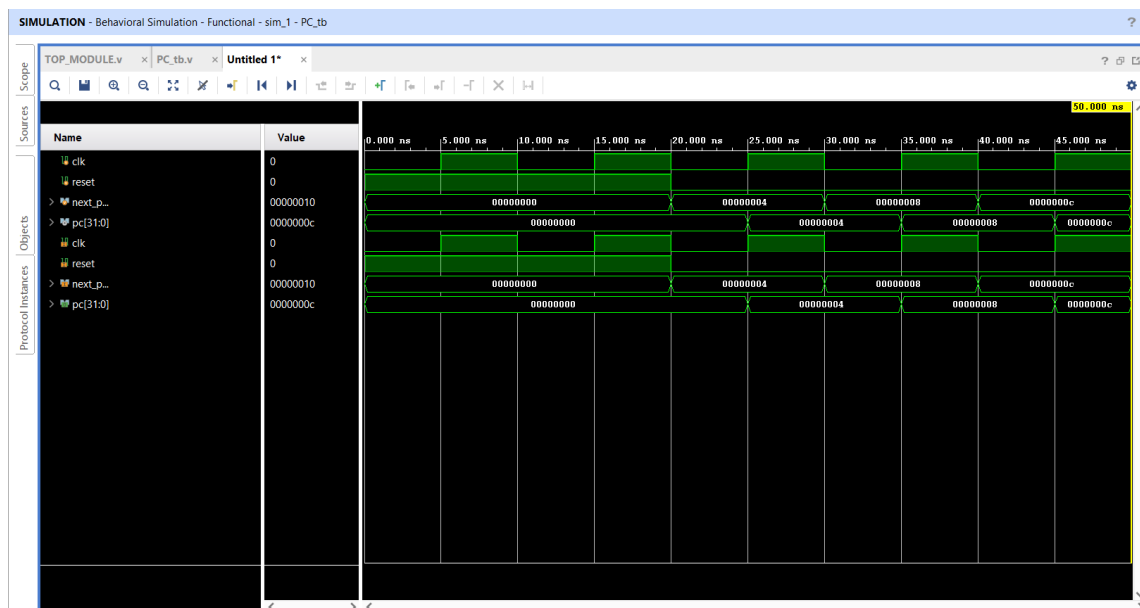
```
next_pc,
```

```
pc);
```

```
end
```

```
endmodule
```

Output Waveform



2. Instruction Memory

Verilog Code:

```
module INSTRUCTION_MEMORY(
```

```
    input  [31:0] address,
```

```
    output [31:0] instruction
```

```
);
```

```
reg [31:0] memory [0:255];
```

```
integer i;
```

```
initial
```

```
begin
```

```

// addi x1, x0, 5
memory[0] = 32'h00500093;
// addi x2, x0, 10
memory[1] = 32'h00A00113;
// add x3, x1, x2
memory[2] = 32'h002081B3;
// sub x4, x2, x1
memory[3] = 32'h40110233;
// sw x3, 0(x0)
memory[4] = 32'h00302023;
// lw x5, 0(x0)
memory[5] = 32'h00002283;
// beq x5, x3, +8
memory[6] = 32'h00328463;
// addi x6, x0, 100
memory[7] = 32'h06400313;
// nop
memory[8] = 32'h00000013;
// Fill remaining memory with NOPs

for(i = 9; i < 256; i = i + 1)
    memory[i] = 32'h00000013;
end

assign instruction = memory[address[31:2]];
endmodule

```

Verilog Test_bench:

```

module INSTRUCTION_MEMORY_tb;
reg [31:0] address;
wire [31:0] instruction;

```

```

INSTRUCTION_MEMORY uut(
    .address(address),
    .instruction(instruction)
);

initial
begin

    address = 32'h00000000; #10;
    address = 32'h00000004; #10;
    address = 32'h00000008; #10;
    address = 32'h0000000C; #10;
    address = 32'h00000010; #10;
    address = 32'h00000014; #10;
    address = 32'h00000018; #10;
    address = 32'h0000001C; #10;
    address = 32'h00000020; #10;

    $finish;

end

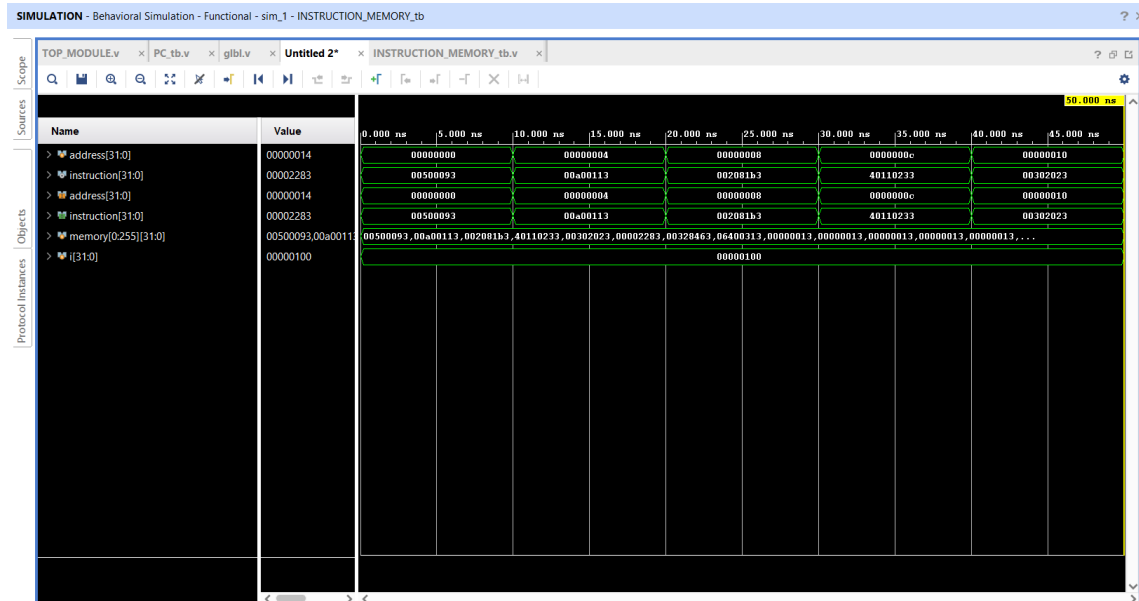
initial
begin
    $display("-----");
    $display("Time\tAddress\t\tInstruction");
    $display("-----");
    $monitor("%0t\t%h\t%h",
        $time,
        address,
        instruction);

end

endmodule

```

Output Waveform



3. Register File

Verilog Code:

```
module REGISTER_FILE(  
    input clk,  
    input RegWrite,  
    input [4:0] rs1,  
    input [4:0] rs2,  
    input [4:0] rd,  
    input [31:0] write_data,  
    output [31:0] read_data1,  
    output [31:0] read_data2  
);  
reg [31:0] registers [0:31];  
integer I;  
initial  
begin  
    for(i = 0; i < 32; i = i + 1)  
        registers[i] = 32'd0;  
end
```

```
assign read_data1 = (rs1 == 5'd0) ? 32'd0 : registers[rs1];
assign read_data2 = (rs2 == 5'd0) ? 32'd0 : registers[rs2];
always @(posedge clk)
begin
    if(RegWrite && (rd != 5'd0))
        registers[rd] <= write_data;
end
endmodule
```

Verilog Test_bench:

```
module REGISTER_FILE_tb;
reg clk;
reg RegWrite;
reg [4:0] rs1;
reg [4:0] rs2;
reg [4:0] rd;
reg [31:0] write_data;
wire [31:0] read_data1;
wire [31:0] read_data2;
REGISTER_FILE uut(
    .clk(clk),
    .RegWrite(RegWrite),
    .rs1(rs1),
    .rs2(rs2),
    .rd(rd),
    .write_data(write_data),
    .read_data1(read_data1),
    .read_data2(read_data2)
);
always #5 clk = ~clk;
initial
```

begin

clk = 0;

RegWrite = 0;

rs1 = 0;

rs2 = 0;

rd = 0;

write_data = 0;

#10;

RegWrite = 1;

rd = 5'd1;

write_data = 32'd5;

#10;

rd = 5'd2;

write_data = 32'd10;

#10;

RegWrite = 0;

rs1 = 5'd1;

rs2 = 5'd2;

#20;

RegWrite = 1;

rd = 5'd0;

write_data = 32'd999;

#10;

RegWrite = 0;

rs1 = 5'd0;

#20;

\$finish;

end

initial

begin

```
$display("-----");

$display("Time\tRegWrite\tRS1\tRS2\tRD\tWrite\tRead1\tRead2");

$display("-----");
```

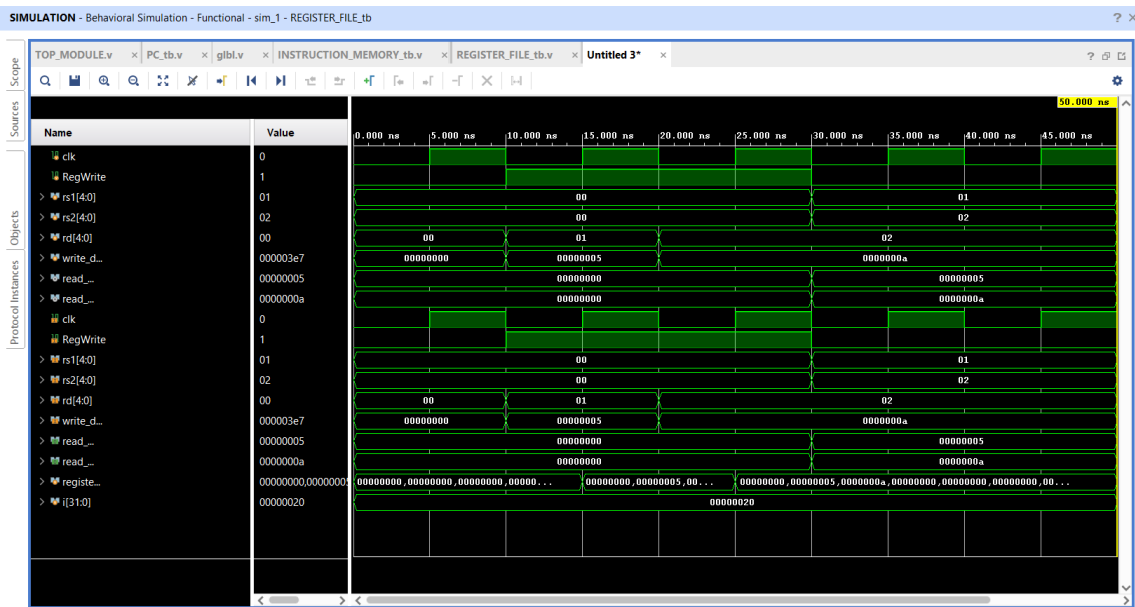
```
$monitor("%0t\t%b\t\t%d\t%d\t%d\t%d\t%d\t%d",
```

```
    $time,
    RegWrite,
    rs1,
    rs2,
    rd,
    write_data,
    read_data1,
    read_data2);

end

endmodule
```

Output Waveform



4 Immediate Generator

Verilog Code:

```
module IMMEDIATE_GENERATOR(
    input [31:0] instruction,
    output reg [31:0] immediate
);
wire [6:0] opcode;
assign opcode = instruction[6:0];
always @(*)
begin
    case(opcode)
        7'b0010011,
        7'b0000011,
        7'b1100111:
        begin
            immediate = {{20{instruction[31]}}},
                        instruction[31:20]};
        end
        7'b0100011:
        begin
            immediate = {{20{instruction[31]}}},
                        instruction[31:25],
                        instruction[11:7]};
        end
        7'b1100011:
        begin
            immediate = {{19{instruction[31]}}},
                        instruction[31],
                        instruction[7],
                        instruction[30:25],
                        instruction[11:8],
```

```

        1'b0};

    end

    7'b0110111,
    7'b0010111:
    begin
        immediate = {instruction[31:12],12'b0};
    end

    7'b1101111:
    begin
        immediate = {{11{instruction[31]}},
                     instruction[31],
                     instruction[19:12],
                     instruction[20],
                     instruction[30:21],
                     1'b0};
    end

    default:
        immediate = 32'd0;
    endcase
end
endmodule

```

Verilog Test_bench:

```

module IMMEDIATE_GENERATOR_tb;
    reg [31:0] instruction;
    wire [31:0] immediate;
    IMMEDIATE_GENERATOR uut(
        .instruction(instruction),
        .immediate(immediate)
    );
    initial

```

begin

instruction = 32'h00500093;

#10;

instruction = 32'h00002283;

#10;

instruction = 32'h00302023;

#10;

instruction = 32'h00328463;

#10;

instruction = 32'h123450B7;

#10;

instruction = 32'h12345097;

#10;

instruction = 32'h0080006F;

#10;

instruction = 32'hFFFFFFFF;

#10;

\$finish;

end

initial

begin

\$display("-----");

\$display("Time\tInstruction\tImmediate");

\$display("-----");

\$monitor("%0t\t%h\t%h",

time,

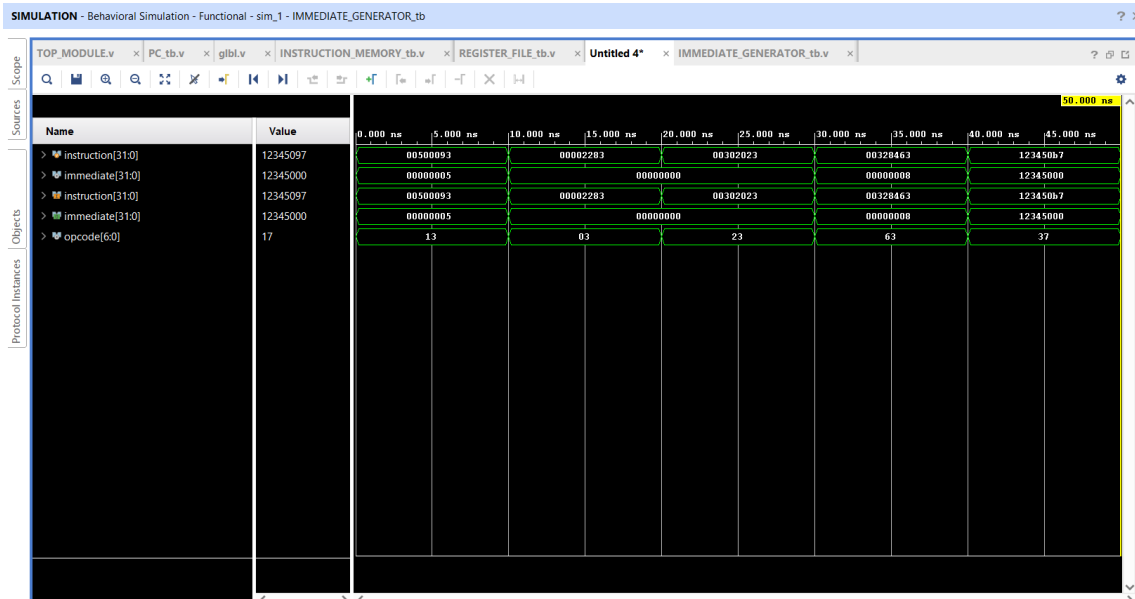
instruction,

immediate);

end

endmodule

Output Waveform



5 Control Unit

Verilog Code:

```
module CONTROL_UNIT(  
    input [6:0] opcode,  
    output reg RegWrite,  
    output reg ALUSrc,  
    output reg MemRead,  
    output reg MemWrite,  
    output reg MemtoReg,  
    output reg Branch,  
    output reg Jump,  
    output reg [1:0] ALUOp  
);  
always @(*)  
begin  
    RegWrite = 0;  
    ALUSrc = 0;  
    MemRead = 0;  
    MemWrite = 0;  
    MemtoReg = 0;
```

```
Branch  = 0;

Jump    = 0;

ALUOp   = 2'b00;

case(opcode)

7'b0110011:

begin

    RegWrite = 1;

    ALUSrc   = 0;

    ALUOp    = 2'b10;

end

7'b0010011:

begin

    RegWrite = 1;

    ALUSrc   = 1;

    ALUOp    = 2'b10;

end

7'b0000011:

begin

    RegWrite = 1;

    ALUSrc   = 1;

    MemRead  = 1;

    MemtoReg = 1;

    ALUOp    = 2'b00;

end

7'b0100011:

begin

    ALUSrc   = 1;

    MemWrite = 1;

    ALUOp    = 2'b00;

end

7'b1100011:
```

```
begin
    Branch = 1;
    ALUOp = 2'b01;
end
7'b0110111:
begin
    RegWrite = 1;
    ALUSrc = 1;
    ALUOp = 2'b11;
end
7'b0010111:
begin
    RegWrite = 1;
    ALUSrc = 1;
    ALUOp = 2'b11;
end
7'b1101111:
begin
    RegWrite = 1;
    Jump = 1;
end
7'b1100111:
begin
    RegWrite = 1;
    Jump = 1;
    ALUSrc = 1;
end
endcase
end
endmodule
```

Verilog Test_bench:

```
`timescale 1ns / 1ps
```

```
module CONTROL_UNIT_tb;
```

```
reg [6:0] opcode;
```

```
wire RegWrite;
```

```
wire ALUSrc;
```

```
wire MemRead;
```

```
wire MemWrite;
```

```
wire MemtoReg;
```

```
wire Branch;
```

```
wire Jump;
```

```
wire [1:0] ALUOp;
```

```
CONTROL_UNIT uut(
```

```
    .opcode(opcode),
```

```
    .RegWrite(RegWrite),
```

```
    .ALUSrc(ALUSrc),
```

```
    .MemRead(MemRead),
```

```
    .MemWrite(MemWrite),
```

```
    .MemtoReg(MemtoReg),
```

```
    .Branch(Branch),
```

```
    .Jump(Jump),
```

```
    .ALUOp(ALUOp)
```

```
);
```

initial

begin

opcode = 7'b0110011; #10; // R-Type

opcode = 7'b0010011; #10; // I-Type

opcode = 7'b0000011; #10; // LW

opcode = 7'b0100011; #10; // SW

opcode = 7'b1100011; #10; // Branch

opcode = 7'b0110111; #10; // LUI

opcode = 7'b0010111; #10; // AUIPC

opcode = 7'b1101111; #10; // JAL

opcode = 7'b1100111; #10; // JALR

opcode = 7'b1111111; #10; // Invalid

\$finish;

end

initial

begin

\$display("-----");

\$display("Opcode RegWrite ALUSrc MemRead MemWrite MemtoReg Branch Jump
ALUOp");

\$display("-----");

\$monitor("%b %b %b %b %b %b %b %b",

opcode,

RegWrite,

ALUSrc,

```

MemRead,

MemWrite,

MemtoReg,

Branch,

Jump,

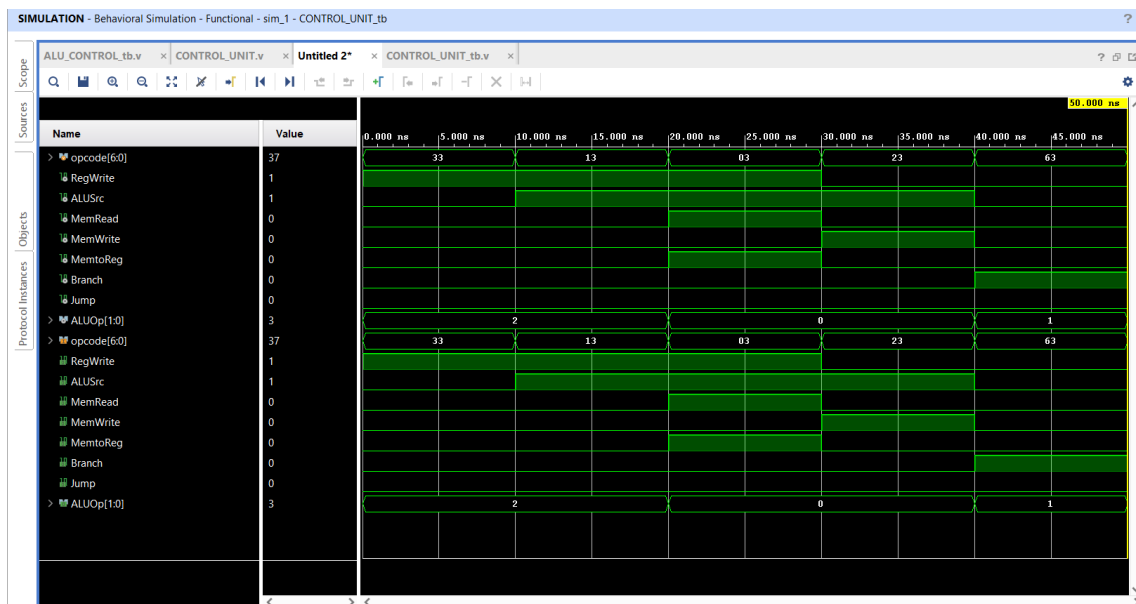
ALUOp);

end

endmodule

```

Output Waveform



6 ALU Control Unit

Verilog Code:

```

module ALU_CONTROL(
    input [1:0] ALUOp,
    input [2:0] funct3,
    input [6:0] funct7,
    output reg [3:0] ALU_Sel
);
always @(*)
begin

```

```
case(ALUOp)
2'b00:
    ALU_Sel = 4'b0000;    // ADD
2'b01:
    ALU_Sel = 4'b0001;    // SUB
2'b10:
begin
    case(func3)
3'b000:
begin
    if(func7 == 7'b0100000)

        ALU_Sel = 4'b0001;    // SUB
    else
        ALU_Sel = 4'b0000;    // ADD
    end
3'b001:
    ALU_Sel = 4'b0101;
3'b010:
    ALU_Sel = 4'b1000;
3'b011:
    ALU_Sel = 4'b1001;
3'b100:
    ALU_Sel = 4'b0100;
3'b101:
begin
    if(func7 == 7'b0100000)
        ALU_Sel = 4'b0111;    // SRA
    else
        ALU_Sel = 4'b0110;    // SRL
```

```
end
    3'b110:
        ALU_Sel = 4'b0011;
    3'b111:
        ALU_Sel = 4'b0010;
    default:
        ALU_Sel = 4'b0000;
    endcase
end
2'b11:
    ALU_Sel = 4'b1010;    // PASS B
default:
    ALU_Sel = 4'b0000;
endcase
end
endmodule
```

Verilog Test_bench:

```
module ALU_CONTROL_tb;
reg [1:0] ALUOp;
reg [2:0] funct3;
reg [6:0] funct7;
wire [3:0] ALU_Sel;
ALU_CONTROL uut(
    .ALUOp(ALUOp),
    .funct3(funct3),
    .funct7(funct7),
    .ALU_Sel(ALU_Sel)
);
```

initial

begin

ALUOp = 2'b00;

funct3 = 3'b000;

funct7 = 7'b00000000;

#10;

ALUOp = 2'b01;

#10;

ALUOp = 2'b10;

funct3 = 3'b000;

funct7 = 7'b00000000;

#10;

funct7 = 7'b01000000;

#10;

funct3 = 3'b001;

funct7 = 7'b00000000;

#10;

funct3 = 3'b010;

#10;

funct3 = 3'b011;

#10;

funct3 = 3'b100;

#10;

funct3 = 3'b101;

funct7 = 7'b00000000;

#10;

funct7 = 7'b01000000;

#10;

funct3 = 3'b110;

```

    funct7 = 7'b00000000;

    #10;

    funct3 = 3'b111;

    #10;

    ALUOp = 2'b11;

    #10;

    $finish;

end

initial

begin

$display("-----");

$display("ALUOp funct3 funct7    ALU_Sel");

$display("-----");

$monitor("%b    %b    %b    %b",

        ALUOp,

        funct3,

        funct7,

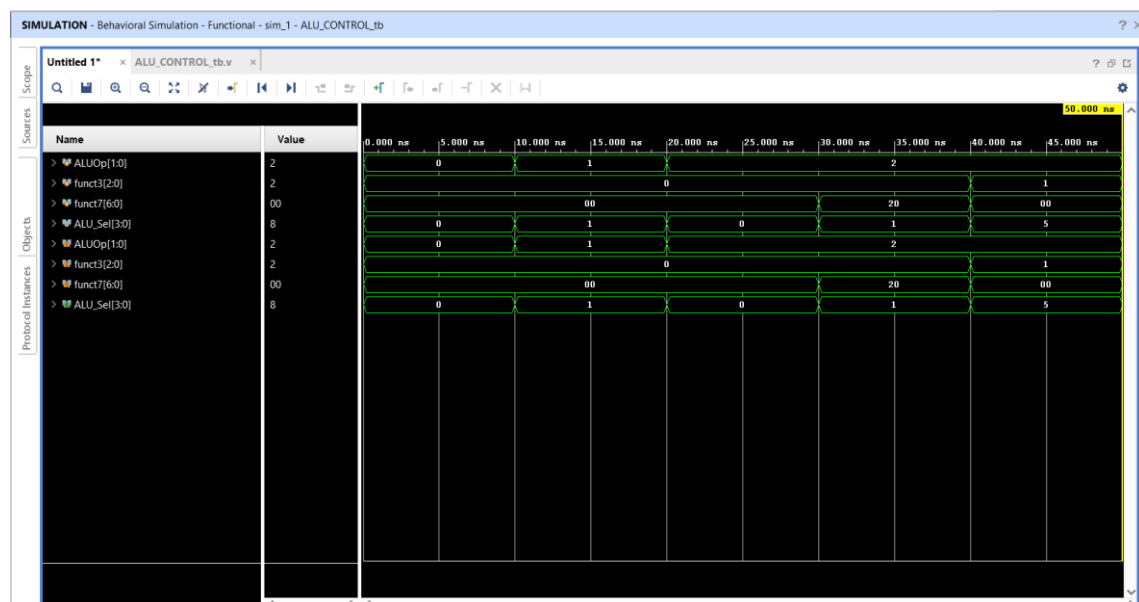
        ALU_Sel);

end

endmodule

```

Output Waveform



7 Arithmetic Logic Unit (ALU)

Verilog Code:

```
module ALU(  
    input [31:0] A,  
    input [31:0] B,  
    input [3:0] ALU_Sel,  
    output reg [31:0] ALU_Result,  
    output Zero  
);  
always @(*)  
begin  
    case(ALU_Sel)  
        4'b0000:  
            ALU_Result = A + B;  
        4'b0001:  
            ALU_Result = A - B;  
        4'b0010:  
            ALU_Result = A & B;  
        4'b0011:  
            ALU_Result = A | B;  
        4'b0100:  
            ALU_Result = A ^ B;  
        4'b0101:  
            ALU_Result = A << B[4:0];  
        4'b0110:  
            ALU_Result = A >> B[4:0];  
        4'b0111:  
            ALU_Result = $signed(A) >>> B[4:0];  
        4'b1000:  
            ALU_Result = ($signed(A) < $signed(B)) ? 32'd1 : 32'd0;
```

```
    4'b1001:
        ALU_Result = (A < B) ? 32'd1 : 32'd0;
    4'b1010:
        ALU_Result = B;
    default:
        ALU_Result = 32'd0;
endcase
end
assign Zero = (ALU_Result == 32'd0);
endmodule
```

Verilog Test_bench:

```
module ALU_tb;
reg [31:0] A;
reg [31:0] B;
reg [3:0] ALU_Sel;
wire [31:0] ALU_Result;
wire Zero;
ALU uut(
    .A(A),
    .B(B),
    .ALU_Sel(ALU_Sel),
    .ALU_Result(ALU_Result),
    .Zero(Zero)
);
initial
begin
    A = 20;
    B = 10;
    ALU_Sel = 4'b0000;
    #10;
```

ALU_Sel = 4'b0001;

#10;

ALU_Sel = 4'b0010;

#10;

ALU_Sel = 4'b0011;

#10;

ALU_Sel = 4'b0100;

#10;

A = 8;

B = 2;

ALU_Sel = 4'b0101;

#10;

A = 32;

B = 2;

ALU_Sel = 4'b0110;

#10;

A = -32;

B = 2;

ALU_Sel = 4'b0111;

#10;

A = 5;

B = 10;

ALU_Sel = 4'b1000;

#10;

A = 5;

B = 10;

ALU_Sel = 4'b1001;

#10;

A = 123;

B = 456;

ALU_Sel = 4'b1010;

```

#10;

A = 50;

B = 50;

ALU_Sel = 4'b0001;

#10;

$finish;

end

initial

begin

$display("-----");

$display("A\tB\tALU_Sel\tALU_Result\tZero");

$display("-----");

$monitor("%d\t%d\t%d\t%d\t%d\t%b",

        A,

        B,

        ALU_Sel,

        ALU_Result,

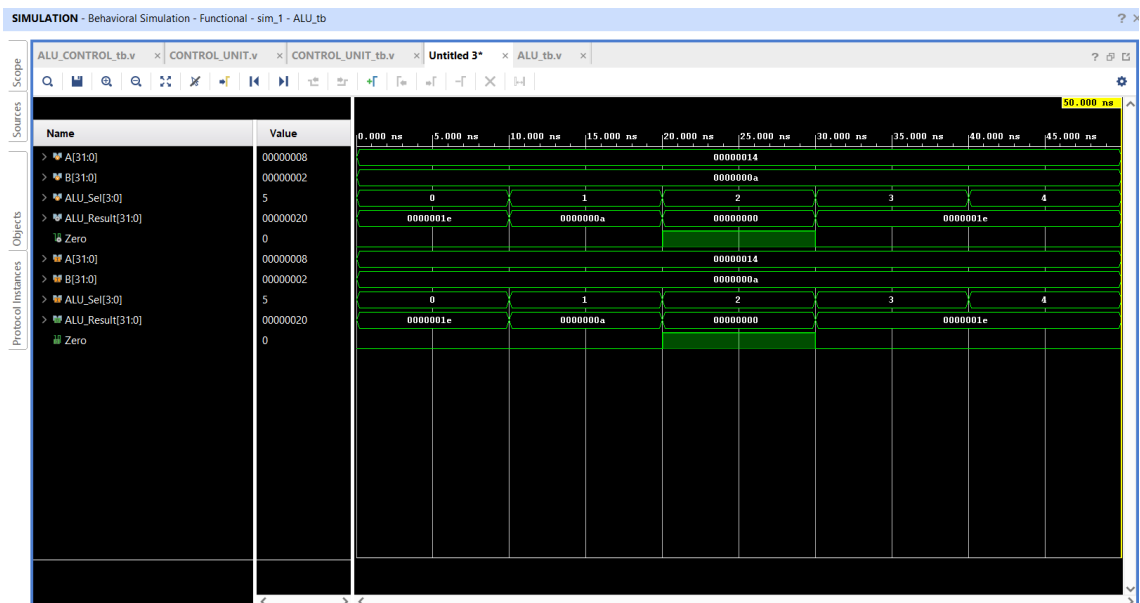
        Zero);

end

endmodule

```

Output Waveform



8 Data Memory

Verilog Code:

```
module DATA_MEMORY(  
    input clk,  
    input MemRead,  
    input MemWrite,  
    input [31:0] address,  
    input [31:0] write_data,  
    output reg [31:0] read_data  
);  
reg [31:0] memory [0:255];  
integer i;  
initial  
begin  
    for(i = 0; i < 256; i = i + 1)  
        memory[i] = 32'd0;  
end  
always @(posedge clk)  
begin  
    if(MemWrite)  
        memory[address[31:2]] <= write_data;  
end  
always @(*)  
begin  
    if(MemRead)  
        read_data = memory[address[31:2]];  
    else  
        read_data = 32'd0;  
end  
endmodule
```

Verilog Test_bench:

```
module DATA_MEMORY_tb;

reg clk;

reg MemRead;

reg MemWrite;

reg [31:0] address;

reg [31:0] write_data;

wire [31:0] read_data;

DATA_MEMORY uut(
    .clk(clk),
    .MemRead(MemRead),
    .MemWrite(MemWrite),
    .address(address),
    .write_data(write_data),
    .read_data(read_data)
);

always #5 clk = ~clk;

initial
begin
    clk = 0;

    MemRead = 0;

    MemWrite = 0;

    address = 32'd0;

    write_data = 32'd0;

    #10;

    MemWrite = 1;

    address = 32'd0;

    write_data = 32'd100;

    #10;

    MemWrite = 0;
```

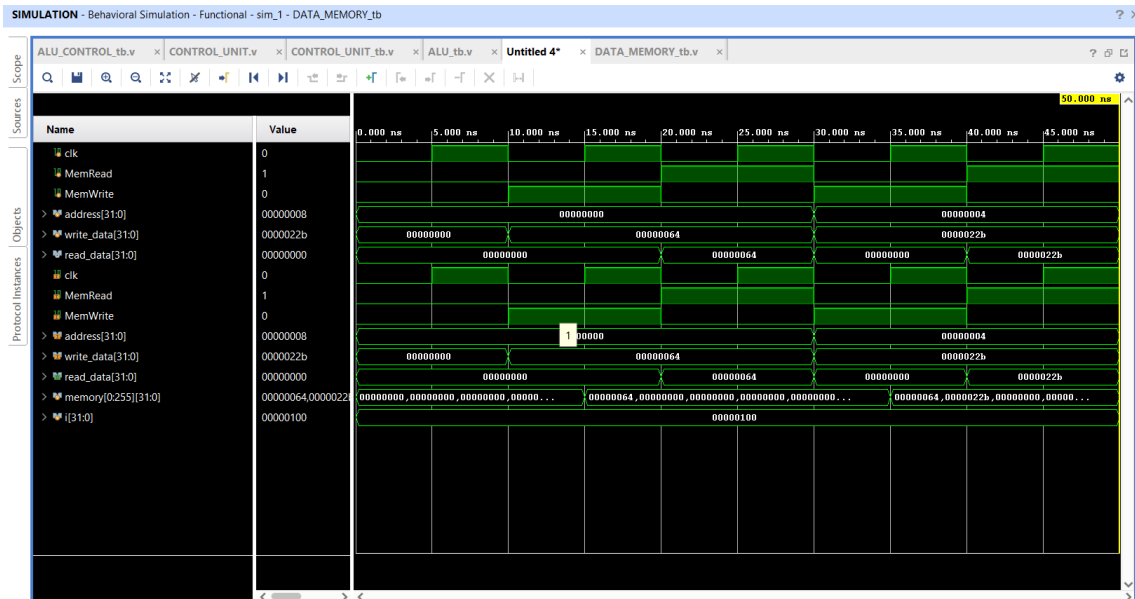
```
MemRead = 1;
address = 32'd0;
#10;
MemRead = 0;
MemWrite = 1;
address = 32'd4;
write_data = 32'd555;
#10;
MemWrite = 0;
MemRead = 1;
address = 32'd4;
#10;
address = 32'd8;
#10;
MemRead = 0;
#10;
$finish;

end

initial
begin
$display("-----");
$display("Time\tAddr\tMemRead\tMemWrite\tWriteData\tReadData");
$display("-----");
$monitor("%0t\t%d\t%b\t%b\t%d\t%d",
    $time,
    address,
    MemRead,
    MemWrite,
    write_data,
    read_data);
end
```

endmodule

Output Waveform



9 Top Module

Verilog Code:

```
module TOP_MODULE(  
    input clk,  
    input reset  
);  
    wire [31:0] pc;  
    wire [31:0] next_pc;  
    wire [31:0] instruction;  
    wire RegWrite;  
    wire ALUSrc;  
    wire MemRead;  
    wire MemWrite;  
    wire MemtoReg;  
    wire Branch;  
    wire Jump;  
    wire [1:0] ALUOp;  
    wire [31:0] read_data1;
```

```
wire [31:0] read_data2;

wire [31:0] write_back_data;

wire [31:0] immediate;

wire [3:0] ALU_Sel;

wire [31:0] alu_input2;

wire [31:0] alu_result;

wire Zero;

wire [31:0] memory_data;

PC PC0(
    .clk(clk),
    .reset(reset),
    .next_pc(next_pc),
    .pc(pc)
);

INSTRUCTION_MEMORY IM0(
    .address(pc),
    .instruction(instruction)
);

CONTROL_UNIT CU0(
    .opcode(instruction[6:0]),
    .RegWrite(RegWrite),
    .ALUSrc(ALUSrc),
    .MemRead(MemRead),
    .MemWrite(MemWrite),
    .MemtoReg(MemtoReg),
    .Branch(Branch),
    .Jump(Jump),
    .ALUOp(ALUOp)
);

REGISTER_FILE RF0(
    .clk(clk),
```

```
.RegWrite(RegWrite),
.rs1(instruction[19:15]),
.rs2(instruction[24:20]),
.rd(instruction[11:7]),
.write_data(write_back_data),
.read_data1(read_data1),
.read_data2(read_data2)
);
IMMEDIATE_GENERATOR IG0(
.instruction(instruction),
.immediate(immediate)
);
ALU_CONTROL AC0(
.ALUOp(ALUOp),
.funct3(instruction[14:12]),
.funct7(instruction[31:25]),
.ALU_Sel(ALU_Sel)
);
assign alu_input2 = (ALUSrc) ? immediate : read_data2;
ALU ALU0(
.A(read_data1),
.B(alu_input2),
.ALU_Sel(ALU_Sel),
.ALU_Result(alu_result),
.Zero(Zero)
);
DATA_MEMORY DM0(
.clk(clk),
.MemRead(MemRead),
.MemWrite(MemWrite),
.address(alu_result),
```

```
.write_data(read_data2),  
.read_data(memory_data)  
);  
assign write_back_data =  
    (MemtoReg) ? memory_data :  
        alu_result;  
assign next_pc =  
    (Jump) ? (pc + immediate) :  
    ((Branch && Zero) ? (pc + immediate) :  
        (pc + 32'd4));  
Endmodule
```

Verilog Test_bench:

```
module TOP_MODULE_tb;  
    reg clk;  
    reg reset;  
    TOP_MODULE uut(  
        .clk(clk),  
        .reset(reset)  
    );  
    always #5 clk = ~clk;  
    initial  
    begin  
        clk = 0;  
        reset = 1;  
        // Apply Reset  
        #20;  
        reset = 0;  
  
        // Run CPU  
        #200;
```

